IBM Machine Learning Professional Certificate

Supervised Machine Learning: Classification - Course Project

A Project Report

on

Comparative Analysis of Classification Models on Heart Disease Dataset

by

Aryan Agarwal

August, 2025

Table of Contents

1. Introduction and Summary of the Dataset

1.1 About the Project

In this project, the aim was to build the best possible supervised machine learning model on a given dataset. We followed a sequential process to build the model, and the steps included preprocessing the data, including cleaning, feature encoding and visualizations to understand the dataset, after which we tested various models such as plain logistic regression, K-nearest neighbors, support vector machines with different kernels and parametrized decision trees; before finally employing various ensemble methods to find the best possible model.

Each model that was built was scored using appropriate scoring metrics including accuracy, precision, recall and F1 scores. These were stored in a separate dataframe, to be analyzed at the end of the project to determine and compare the performances of different models.

1.2 Information about the Dataset

For this project, the dataset chosen was the Heart Disease Dataset, available on Kaggle. This dataset lists medical information of patients admitted to various hospitals with complains of chest pain.

The information available include personal details such as the age, gender as well as medical details such as the type of chest pain, heart rate, blood pressure, blood sugar etc.

The dataset also includes a column specifying whether the patient has heart disease or not, and this feature will serve as our target variable in this project. Our goal shall be to build a model that can best predict this feature.

The dataset was imported from the following URL:

https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction

1.3 Additional Information

This analysis was performed on the web-based Jupyter Notebook IDE, using the Python (Pyodide) kernel.

The coding was done in Python 3.13.3, employing the following libraries for various steps and processes:

- Pandas
- NumPy
- Matplotlib
- Seaborn
- SciPy
- Scikit-learn

The following functions of the scikit-learn library were used in this project:

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedShuffleSplit
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LogisticRegressionCV
from sklearn.metrics import precision_recall_fscore_support as prf_score
from sklearn.metrics import confusion_matrix, accuracy_score, f1_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import GridSearchCV
```

1.4 Dataset Summary

The dataset spans 918 rows, each representing a single patient, and 12 columns, each representing a certain detail about the patient. It consists of 5 categorical (object) and 7 numerical (int, float) columns, as follows:

```
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    object
 2   ChestPainType   918 non-null    object
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    object
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    object
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), object(5)
```

The numeric columns have the following data distribution:

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364	0.553377
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570	0.497414
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000	0.000000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000	1.000000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000	1.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000	1.000000

2. Data Cleaning and Preprocessing

2.1 Cleaning and Feature Encoding

Before training any models on our dataset, the raw data was processed to ensure that it is fit to be passed into the various different models to be trained on it ahead.

As was seen in the previous section, there are no null values in any of the columns, so there was no need to perform any cleaning to fill or remove any data points.

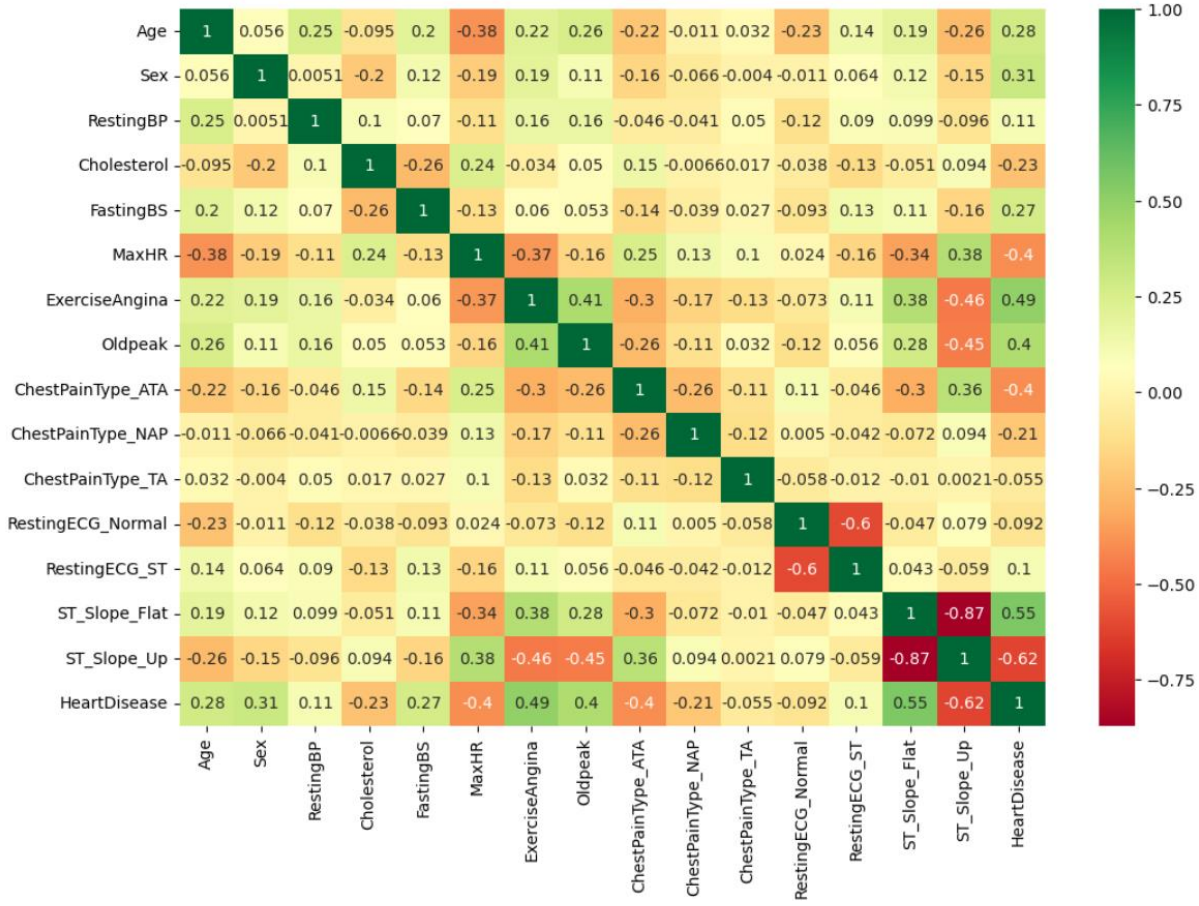
Next, we can see that 5 of the 12 feature columns are of object datatype. To be able to use them in our model, these were encoded using standard encoding methods. The number of unique values in each of the categorical columns was as follows:

```
Sex              2
ChestPainType    4
RestingECG       3
ExerciseAngina   2
ST_Slope         3
```

Among these, the binary columns i.e. columns having only two unique values were simply mapped to 0s and 1s, while the remaining columns were label encoded using the .get_dummies() function of Pandas. The final dataset contained 15 feature columns and 1 target variable.

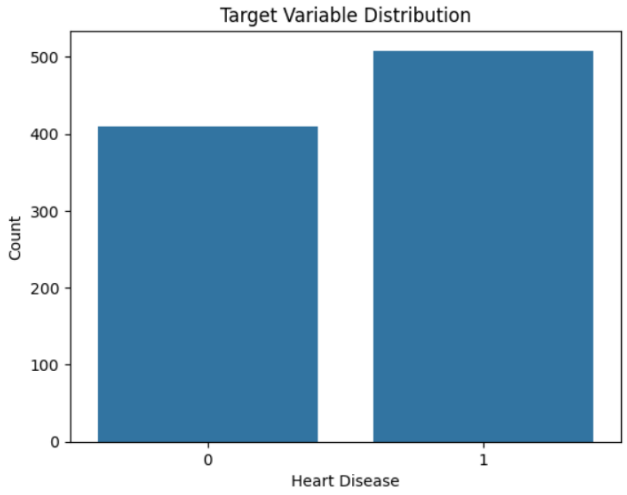
2.2 Visualizations

The next step was to visualize the correlations between different features, using the .heatmap() function of Seaborn:



As we can see from the above heatmap, the target variable seems to be most highly correlated with the features ‘ExerciseAngina’ and ‘ST_Slope’, and in general has appreciable correlation with other features as well.

Next, we also visualized the distribution of classes within the target variable ‘HeartDisease’ to determine whether we would be working with balanced or imbalanced classes, and to accordingly tune the models:



As can be seen, the classes are pretty well-balanced, with only a slightly higher number of positive cases than negative ones. While the imbalance is slight, we have nevertheless used a stratified split to mitigate any potential issues that might arise due to class imbalance.

2.3 Data Preparation

Next, we separated our feature columns and target columns into different dataframes, and performed a stratified train-test-split on these, keeping the test set size at 25%.

After this, the entire dataset was scaled using the StandardScaler() function, to ensure that models such as k-NN and SVMs would not be affected by feature scale.

Subsequently, we implemented the scoring system for the models by defining two functions, one to calculate and store the aforementioned metrics for each model, and one to plot a confusion matrix for each model. Furthermore, an empty dataframe to store these scores was also created.

Lastly, an integer variable was initialized to store the value for the 'random_state' parameter for all of our models.

With this, the preprocessing steps were complete, and the dataset was ready to train the first model.

3. Logistic Regression

3.1 Basic Model

The first model that we built was a simple logistic regression model using the LogisticRegression() function of scikit-learn. The solver used was the default solver 'lbfgs', with no regularization penalty.

This model yielded the following F1 score:

```
F1 score for Logistic Regression   -   0.896551724137931
```

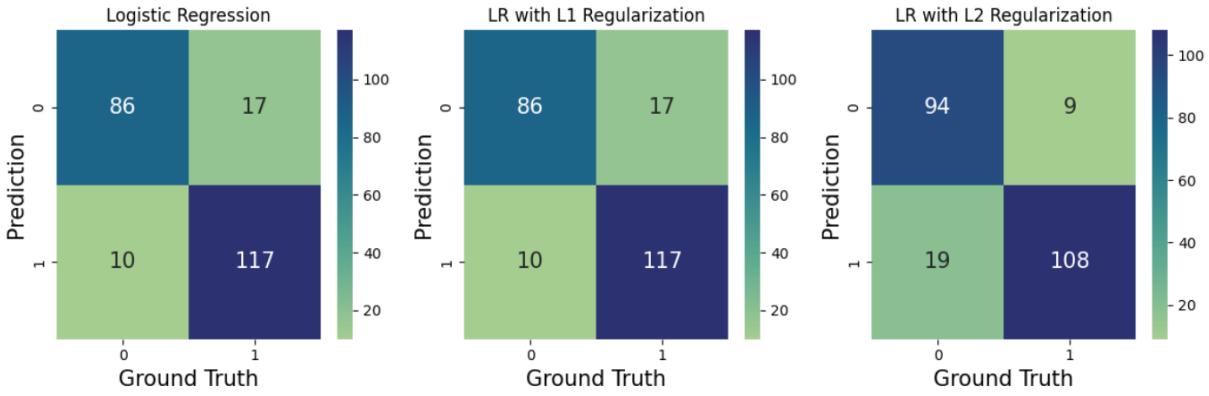
3.2 Regularized Models

The next models we trained were built using the LogisticRegressionCV() function, using the 'liblinear' solver with multiple C values and a 3-fold split. One model each was trained using the L1 and L2 methods of regularization.

These yielded the following F1 scores:

```
F1 score for LR with L1 Regularization   -   0.896551724137931
F1 score for LR with L2 Regularization   -   0.8852459016393442
```

The confusion matrices for all three logistic regression models are shown below:



4. K-Nearest Neighbors

4.1 Basic Model

Next up, we trained a model using the method of K-nearest neighbors. This was achieved using the KNeighborsClassifier() function, with the K value set to 3.

This model yielded the following F1 score:

```
F1 score for K-Nearest Neighbors  -  0.90625
```

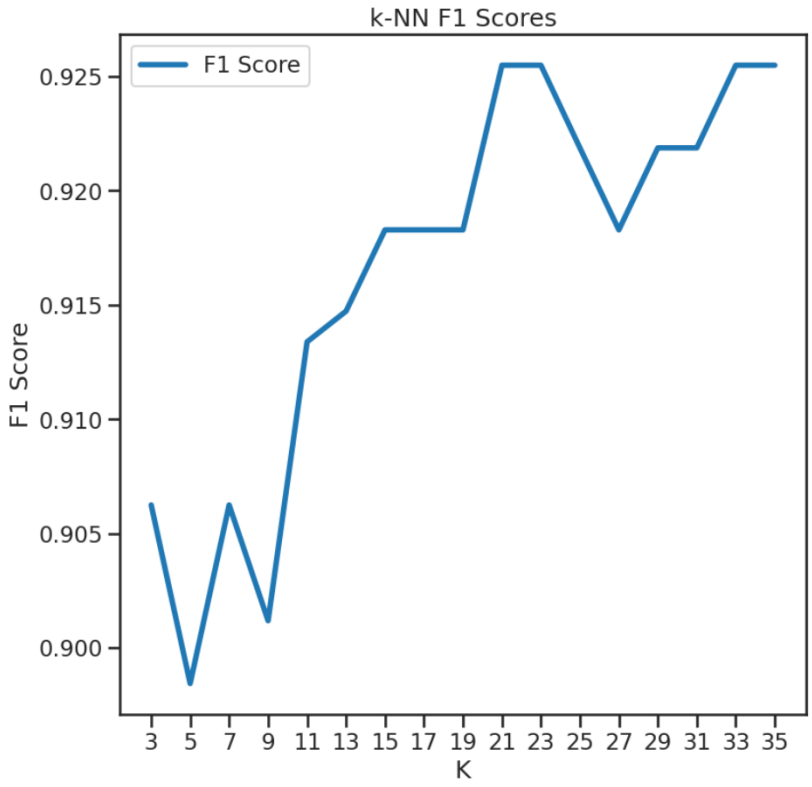
4.2 Tuned Model

Then, for the next model, the value of K was varied by iterating through all the odd numbers between 3 and 35, using a for-loop, to find the value of K for which the model would perform the best.

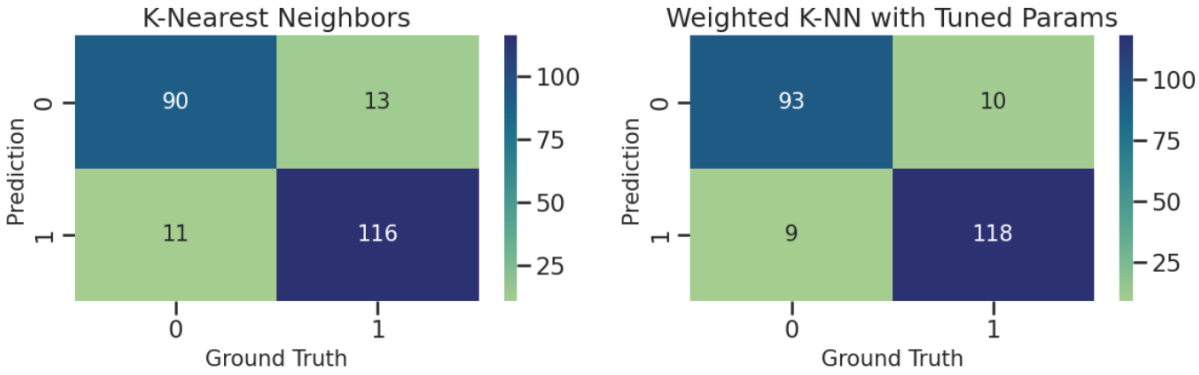
The best K value and the corresponding F1 score were as follows:

```
Best K =  21
F1 score for Weighted k-NN with Tuned Params  -  0.9254901960784314
```

The graph of F1 scores across different K values is shown below:



The confusion matrices for the basic and tuned models were as follows:



5. Support Vector Machines

5.1 Basic Model

Moving on, we next trained a model using the Support Vector Machine classifier. The basic model was created using the LinearSVC() function, with no parameters tuned.

This model yielded the following F1 score:

```
F1 score for Linear Support Vector Machine  -  0.9
```

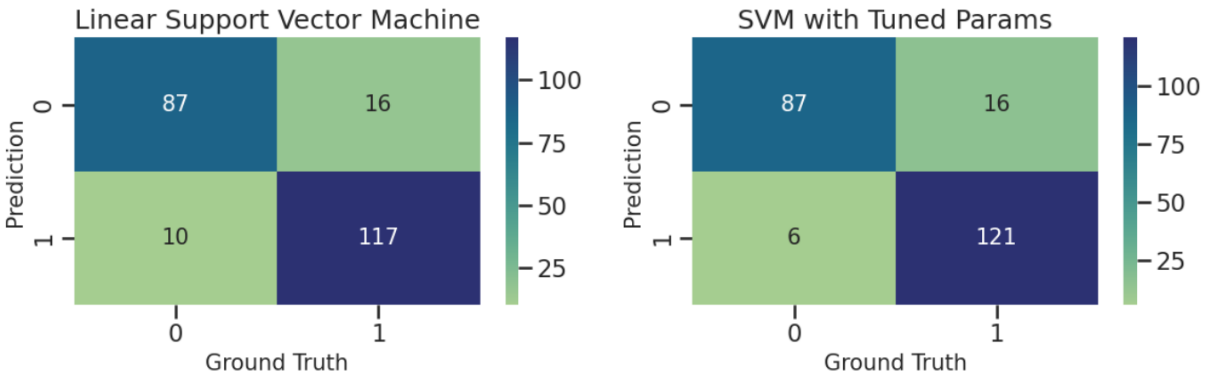
5.2 Tuned Model

Thereafter, to tune the model, the SVC() function was used with the Gaussian kernel 'rbf'. The values of 'gamma' and 'C' were varied by iterating through two nested for-loops, to find the best values.

The best parameters and the corresponding F1 score were as follows:

```
Best params:  [0.1, 1]
F1 score for SVM with Tuned Params  -  0.9166666666666666
```

The confusion matrices for the basic and tuned models were as follows:



6. Decision Trees

6.1 Basic Model

After SVMs, the next method used was a Decision Tree. This model was created by using the DecisionTreeClassifier() function, with no parameters tuned..

This model yielded the following F1 score:

```
F1 score for Decision Tree   -   0.8221343873517787
```

6.2 Tuned Model

Thereafter, to tune the basic model, the GridSearchCV() function was used to perform cross-validation and find the best parameters from a predefined parameter grid. The parameters 'max_depth' and 'max_features' were both tuned by iterating them through different values:

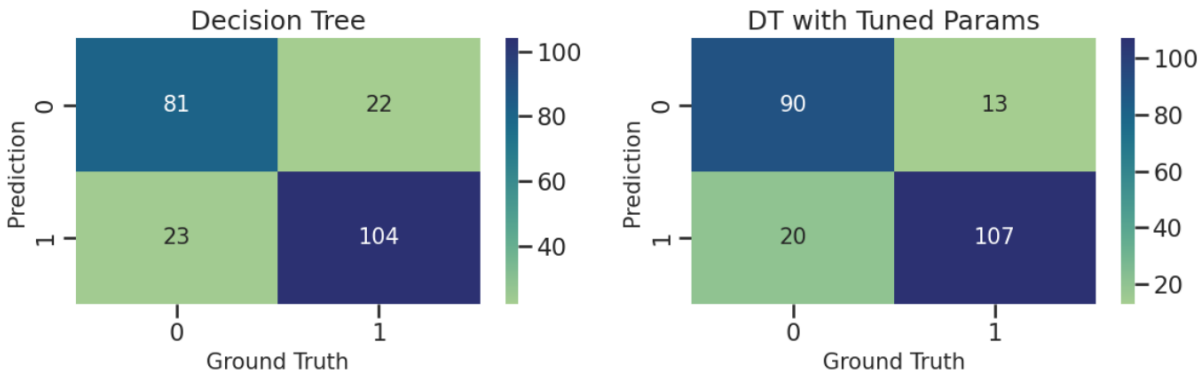
- 'max_depth' was iterated from 3 to the maximum depth of the original tree.
- 'max_features' was iterated from 3 to 11.

The best parameters and the corresponding F1 score were as follows:

```
{'max_depth': 3, 'max_features': 9}

F1 score for DT with Tuned Params   -   0.8663967611336032
```

The confusion matrices for the basic and tuned models were as follows:



7. Ensemble Methods

7.1 Section Overview

Finally, various ensemble methods were utilized to combine multiple iterations of the same base model, or different models, in order to use the best features of each.

The rationale was that if different models made complementary mistakes on different data points, such ensemble models could analyse the performance of different models and learn from them to create a better meta-model that would avoid such mistakes.

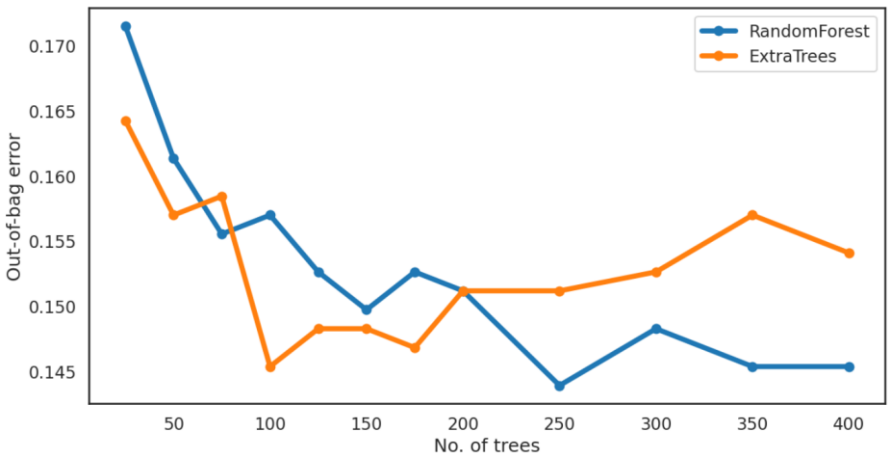
7.2 Random Forest and Extra Random Trees

The first ensemble models used were the Random Forest and Extra Random Trees classifiers, both of which employ a multitude of different decision trees, each trained on different subsets of the data, a process known as bootstrap aggregating or 'bagging'. Then the predictions from each tree are used to make the final prediction.

While Random Forests train each tree on the entire dataset, Extra Random trees go one step further and use random split points as well as a random subset of data points on which to train the model.

These models were created using the RandomForestClassifier() and ExtraTreesClassifier() methods respectively. Each model was iterated through a range of values for the number of trees to be used. Subsequently, the models were scored using the out-of-bag errors, which were stored in a separate list.

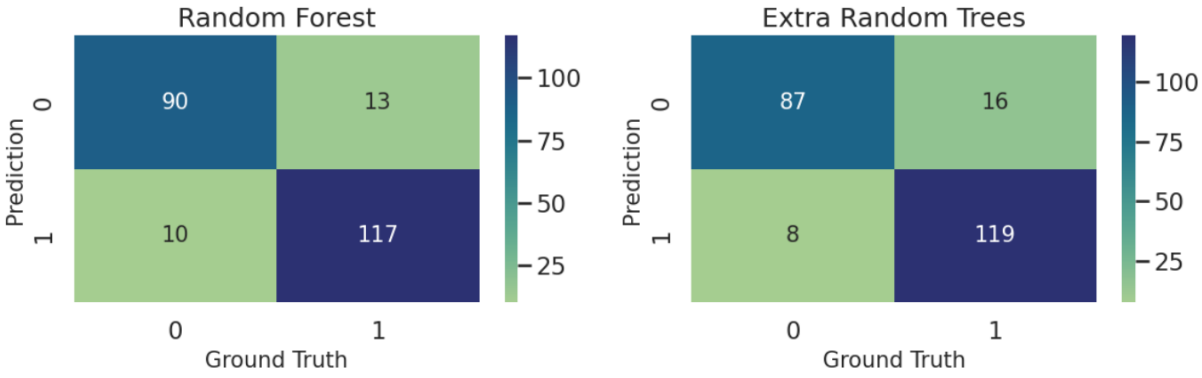
The graph of out-of-bag errors for both models are as follows:



The F1 scores for each model using the best parameters were as follows:

```
F1 score for Random Forest  -  0.9105058365758755
F1 score for Extra Random Trees  -  0.9083969465648855
```

The confusion matrices for these models were as follows:



7.3 Gradient Boosting and Adaptive Boosting

The next ensemble methods we built were the Gradient Boosting and Adaptive Boosting classifiers, created using the GradientBoostingClassifier() and AdaBoostClassifier() methods respectively.

These models work by sequentially creating weak learners like decision trees, with each new tree trained on the model as well as previous trees, and using the final tree as the predictor, effectively improving the model in each iteration. Both models use the concept of gradient descent, and attempt to minimize a loss function.

The key difference is that AdaBoost puts more emphasis on misclassified data points by weighting them higher in later trees, and typically uses trees with only a depth of only 1, essentially a stump.

Both these models were tuned using GridSearchCV(), with predefined parameter grids.

The parameter grid for the gradient boosting classifier included the features ‘learning_rate’, ‘max_features’, ‘n_estimators’ and ‘subsample’. The best parameters and F1 score were as follows:

```
param_grid_grad = {'n_estimators': [150,200,300],
                   'max_features': [3,4,5],
                   'learning_rate': [0.001,0.01,0.1],
                   'subsample': [1.0,0.8]}
```

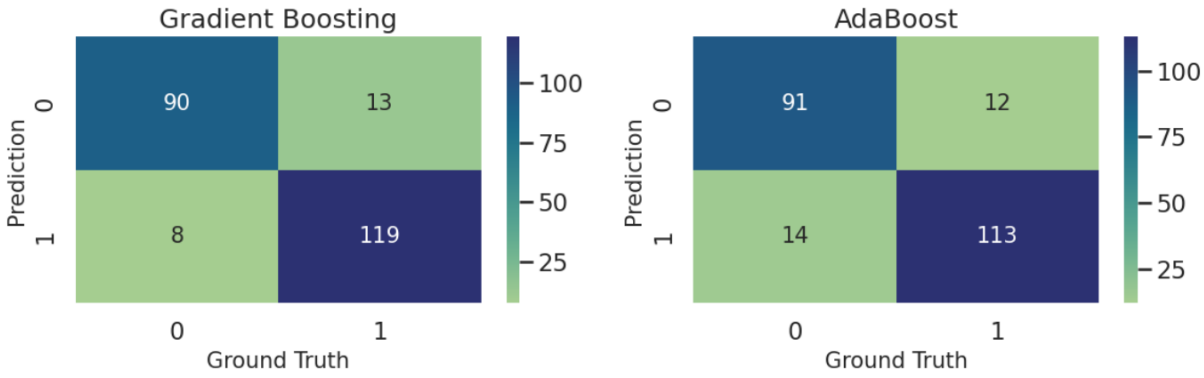
The parameter grid for the adaptive boosting classifier included the features ‘learning_rate’, ‘max_features’, ‘n_estimators’. The best parameters and F1 score were as follows:

```
param_grid_ada = {'n_estimators': [50,80,100,150,200],
                  'learning_rate': [0.1,0.5,1.0]}
```

The F1 scores for both of these models are as follows:

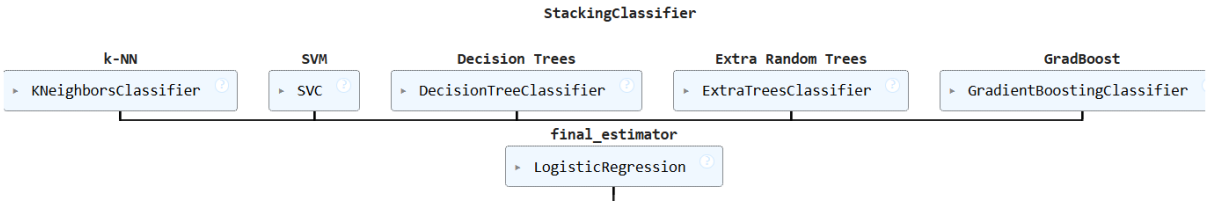
```
F1 score for Gradient Boosting  -  0.918918918918919
F1 score for AdaBoost  -  0.8968253968253969
```

The confusion matrices for these models were as follows:



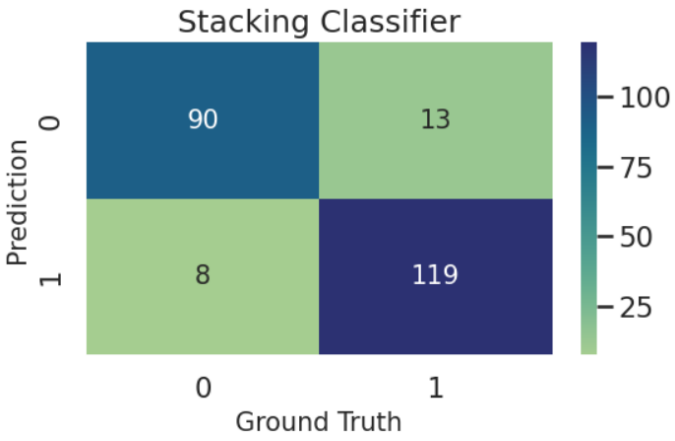
7.4 Stacking Classifier

Finally, we used the stacking classifier model to combine various previous models, using their tuned hyperparameters. The model was created using the StackingClassifier() function. Five different models were used as base estimators, with Logistic Regression serving as the meta-model and final estimator:



The final F1 score and confusion matrix for this model were as follows:

```
F1 score for Stacking  -  0.918918918918919
```



8. Key Findings and Insights

8.1 Preprocessing and EDA Findings

The progressive more refined models built on the dataset provided some key insights about the data itself as well as the relationships present within the data, both, between different features, as well as between the features and target variable.

While preprocessing, we saw that the dataset was a clean dataset with no missing values and a wide range of features, both numerical and categorical, with the latter being easily encodable. Furthermore, the target variable was decently well-balanced between both classes, making it simpler to train models on it without worrying about accounting for class imbalance.

A correlation heatmap of the entire dataset showed that the features of 'ExerciseAngina' and 'ST_slope' had the highest absolute correlation with the target feature 'HeartDisease'.

8.2 Model Performance Summary

Thereafter, a total of 14 different models were trained and tested on this dataset. These models were scored on multiple metrics, the results of which are displayed below:

	Model	Accuracy	Precision	Recall	F1
0	Logistic Regression	0.882609	0.873134	0.921260	0.896552
1	LR with L1 Regularization	0.882609	0.873134	0.921260	0.896552
2	LR with L2 Regularization	0.878261	0.923077	0.850394	0.885246
3	K-Nearest Neighbors	0.895652	0.899225	0.913386	0.906250
4	Weighted k-NN with Tuned Params	0.917391	0.921875	0.929134	0.925490
5	Linear Support Vector Machine	0.886957	0.879699	0.921260	0.900000
6	SVM with Tuned Params	0.904348	0.883212	0.952756	0.916667
7	Decision Tree	0.804348	0.825397	0.818898	0.822134
8	DT with Tuned Params	0.856522	0.891667	0.842520	0.866397
9	Random Forest	0.900000	0.900000	0.921260	0.910506
10	Extra Random Trees	0.895652	0.881481	0.937008	0.908397
11	Gradient Boosting	0.908696	0.901515	0.937008	0.918919
12	AdaBoost	0.886957	0.904000	0.889764	0.896825
13	Stacking	0.908696	0.901515	0.937008	0.918919

As can be seen from the above table, the best performing model out of all 14, based on the F1 score, was the 'K-Nearest Neighbours' classifier, with a tuned K-value and weighted data points, with an F1 score of 0.925490.

8.3 Key Takeaways

This finding can be used to make multiple inferences about our data and other models:

- Firstly, the classes of our target variable might be clustered, which would make the k-NN model highly effective in correctly predicting a data point's class by using its neighbours.
- Secondly, our dataset most likely has a non-linear decision boundary, since other linear models such as logistic regression underperformed.
- Thirdly, the performance of the k-NN model suggests that the feature space is not too high-dimensional.

The ensemble models of gradient boosting and stacking classifiers were the next best performers with an identical F1 score of 0.918919.

Among the other metrics, the tuned k-NN model also had the highest accuracy of 0.917391. However, the L2-regularized logistic regression model had the highest precision at 0.923077, and the highest recall was that of the tuned support vector machine model with the Gaussian kernel, clocking in an impressive recall score of 0.952756.

9. Conclusion

9.1 Summary

In conclusion, this project proved to be an insightful and comprehensive deep-dive into exploring different classifier models, how they are implemented, and their strong and weak points. Furthermore, it also revealed key insights about the Heart Disease dataset, and the relationships and correlations between its features.

In this project, we started off by choosing and importing the Heart Disease dataset, along with all the libraries that would be used throughout the project. Then the dataset was pre-processed, which involved encoding categorical features, scaling the dataset, and visualizing the correlations between the features with a heatmap. Then, the dataset was split into train and test sets, and the first model was trained on it. Over the course of the project, 14 different models were trained including the basic and tuned versions of logistic regression, K-nearest neighbours, support vector machines, decision trees, and ensemble models such as random forest, extra trees, boosting models and finally, the stacking classifier. Lastly, all models were compared using their F1 scores and other metrics, and key insights were presented.

9.2 Next Steps

With the modelling and analysis now complete, the possible next steps that we can take include:

- Using more advanced techniques to improve model performance, such as more complex ensemble methods, with better regularization methods.
- Building up a pipeline to streamline the model building process, and add support for any other functionalities or processes that might be added in a real-world implementation of such a model-building project.
- Performing deeper analysis of the best models to understand why they were better than the others, by using tools such as SHAP or LIME to explain predictions.

These steps will ensure that the takeaways from this modelling project are optimally used, and prove beneficial in practical applications further down the line.